

HOSTELPOCKET

Student Hostel Personal Expense Tracker

Final Workshop Project Documentation

STUDENT	REGISTER NO.	SCHOOL / PROGRAM
Javin Kapur	26BCE0024	SCOPE (CSE Core)

PROJECT SNAPSHOT

A Streamlit-based student finance application that combines durable PostgreSQL expense storage, item-level purchase history, budget meters, AI-assisted expense capture, reminders, activity timeline, and safe sample UPI payment shortcuts.

LIVE WEBSITE	https://hostel-pocket-breakingbytes.onrender.com
GITHUB REPOSITORY	https://github.com/JavinKapur/Hostel_Pocket_BreakingBytes

Prepared for workshop submission • 21 September 2026

1. Project Overview

HostelPocket is a student-focused personal expense tracker designed around a common hostel problem: small, frequent purchases are easy to forget and difficult to understand later. The current public implementation combines a Streamlit interface with a PostgreSQL-backed data layer, AI-assisted capture workflows, and lightweight payment shortcuts for friend-related payments.

CORE VALUE

Capture spending quickly -> save it reliably -> understand where the money went -> act before the budget is exhausted.

Objectives

- Make daily expense capture fast enough for real hostel-life transactions.
- Persist both the expense total and the purchased item lines so past history remains useful.
- Replace roommate-centric flows with a simple Friends model for optional context and payment shortcuts.
- Give students both daily and monthly budget visibility through circular utilization meters.
- Use AI where it removes friction: natural-language parsing, voice entry, receipt OCR, and financial insights.
- Keep payment functionality demonstrative and safe by generating sample UPI URIs and QR codes rather than processing real transactions.

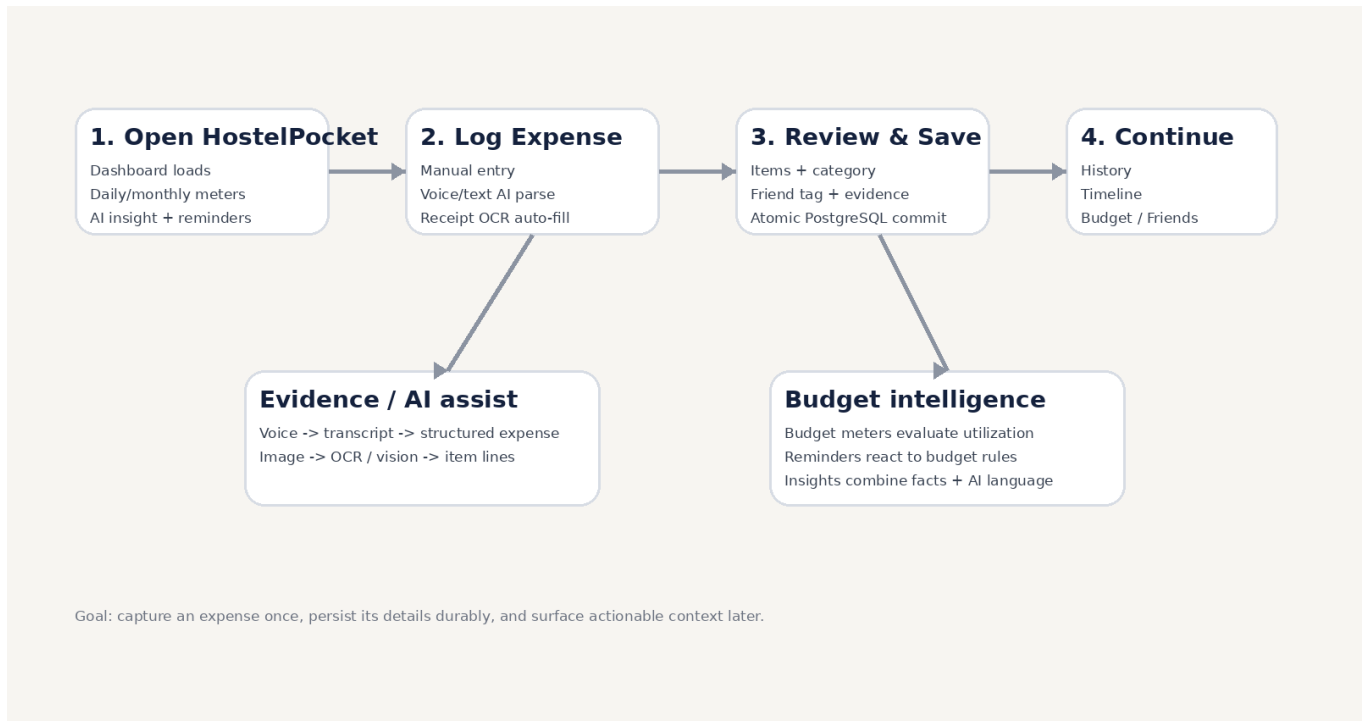
2. Complete Feature Catalogue

The following catalogue reflects the feature set visible in the public repository implementation, with the user-facing modules cross-checked against the main Streamlit application and repository README.

Module / Feature	What the user can do	Implementation detail
Dashboard Daily & monthly budget meters	Two circular meters show spent versus budget for today and the current month.	Budget repository + circular SVG meter component.
Dashboard Quick actions	Direct buttons open Add Expense, History, Timeline, Budgets, and Friends.	Centralized route stack and Streamlit buttons.
Dashboard Category drilldown	Each spending category can be expanded to reveal exact item rows, quantity, and total spend.	Item-level aggregation from persisted expense items.
Dashboard Friends payment shortcuts	Up to four friends can be surfaced directly with a Show QR action.	Friend records + UPI URI/QR rendering.
Dashboard AI insight strip	Latest persisted financial insight is shown with a refresh action.	Deterministic aggregates + Groq refinement + insight persistence.
Dashboard Active reminders	Pending reminders are evaluated against current budget data and can be dismissed.	Reminder repository + budget rule evaluation.
Add Expense Manual expense entry	Merchant/title, total amount, category, date, optional friend, notes, item lines, and evidence can be recorded.	Streamlit form + repository service layer.
Add Expense Voice expense capture	Student can record a sentence such as a dinner payment; audio is transcribed and parsed into structured fields.	Groq Whisper with SpeechRecognition fallback.

Module / Feature	What the user can do	Implementation detail
Add Expense Natural-language expense parsing	Freeform text is converted into title, category, amount, friend, and item lines.	Groq text model with heuristic fallback.
Add Expense Receipt / bill OCR	Image evidence can be scanned to auto-fill vendor, category, total, and item lines.	Vision-capable Groq model -> Tesseract/OpenCV fallback -> LLM parse.
Add Expense Unified evidence workflow	Receipt photo or delivery screenshot is handled through one evidence uploader/banner instead of separate receipt/screenshot concepts.	Single evidence component + attachment metadata.
History Persistent expense history	Saved expenses appear in a durable PostgreSQL history rather than only transient UI state.	Expense repository queries persisted rows.
History Search & filters	Search text across title/note/item context; filter by category and friend.	Repository-level filtering with a 50-record UI window.
History Purchased item ledger	An expense can be expanded to show item name, quantity, unit price, line total, and evidence metadata.	Normalized expense_items + attachment metadata.
Timeline Activity timeline	Append-only events show what happened, the event type, timestamp, and structured payload.	timeline_events persisted in PostgreSQL.
Budgets Daily budget setter	Set or update the current day budget.	Budget repository + timeline event on update.
Budgets Monthly budget setter	Set or update the current month budget.	Budget repository + timeline event on update.
Friends Named friends model	Store friend name, sample UPI ID, and a relationship/context note.	Friends repository; no roommate UI concept.
Friends Sample UPI QR	Generate a QR code with payee and optional prefilled amount.	UPI URI builder + QR generation.
Friends App deep links	Produce standard UPI plus Google Pay, PhonePe, Paytm, and BHIM/generic shortcuts.	App-specific URI schemes.
Navigation Deterministic back button	Back pops the navigation stack; when empty it falls back to dashboard.	Centralized navigate_to / navigate_back functions.
UI Student-friendly design system	Wide layout, sidebar navigation, custom CSS, rounded cards, status badges, and display/body typography.	Streamlit CSS tokens and reusable components.
Observability PRISM telemetry	AI and pipeline calls are instrumented with model/agent and latency information.	PRISM tracer integration.

3. Major User Workflows



3.1 Manual expense workflow

1. Open Add Expense from the dashboard or sidebar.
2. Enter title/merchant, total amount, category, date, optional friend, note, and item lines.
3. Optionally attach a receipt photo or delivery screenshot through the unified evidence area.
4. Submit the form; the application validates item totals and performs an atomic persistence operation.
5. On success, the application navigates to History so the saved record can be immediately verified.

3.2 AI-assisted expense workflow

6. Provide a freeform sentence or a short voice recording.
7. The AI layer transcribes audio when required, then parses the expense into structured JSON-like fields.
8. The parsed values prefill the standard expense form so the student remains in control of the final record.
9. The same persistence path is used for the final save, preserving item lines and timeline information.

3.3 Receipt scanning workflow

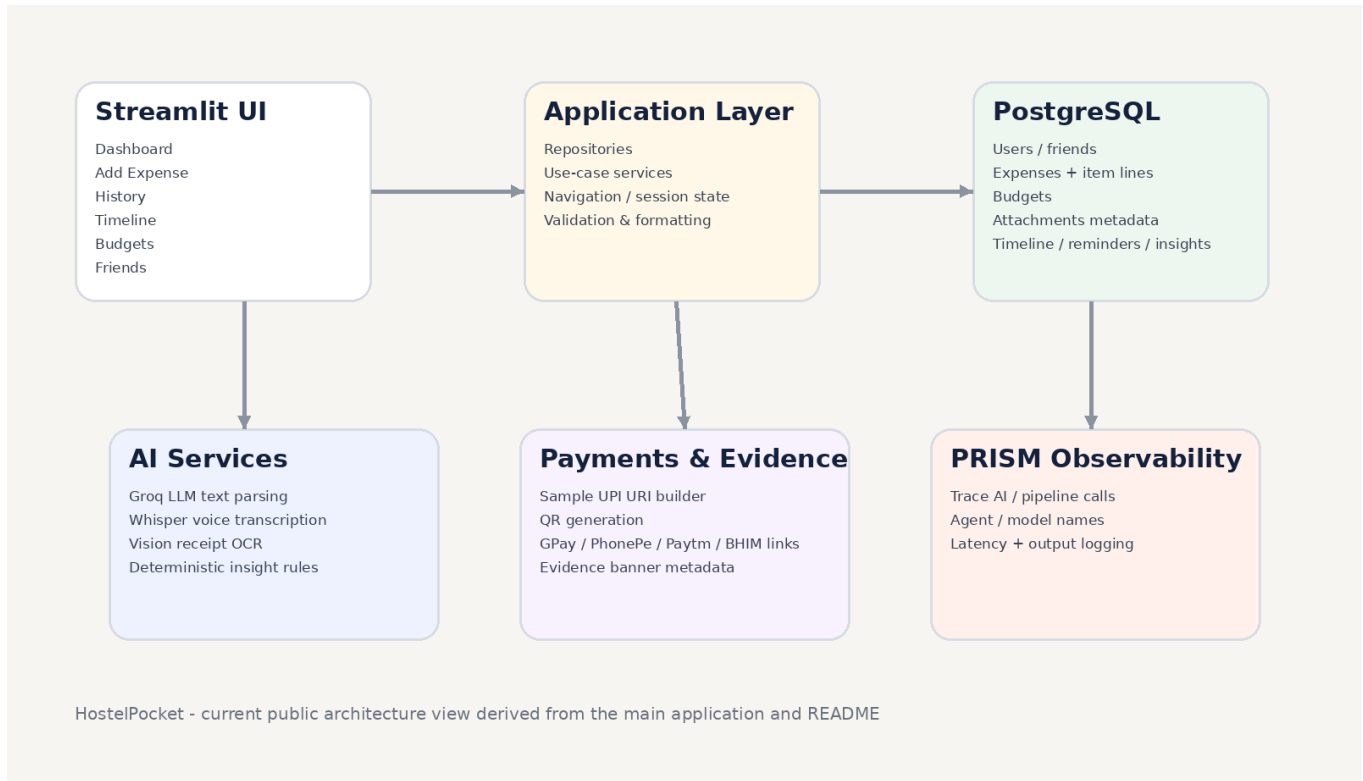
10. Upload a PNG/JPG bill or delivery-order screenshot.
11. Run OCR/vision extraction to identify vendor, category, total, and item lines.
12. Review the auto-filled data and correct any low-confidence field before saving.
13. Persist the final expense together with attachment metadata.

3.4 Friend payment workflow

14. Create a Friend record with a sample UPI ID and optional context note.
15. From the Dashboard or Friends screen, generate a QR code for the selected friend.
16. The QR contains a sample UPI payment URI with payee and prefilled amount.

17. The app can expose standard UPI and app-specific deep links for common Indian UPI apps.

4. Technical Architecture



Architecture layers

- Presentation layer - Streamlit screens and reusable UI components provide the dashboard, expense form, history, timeline, budgets, and friend-management views.
- Application/service layer - repositories and services separate UI concerns from persistence, insight generation, reminders, QR construction, validation, and navigation.
- Persistence layer - PostgreSQL stores user-scoped expenses, item lines, friends, budgets, attachment metadata, timeline events, reminders, and insights.
- AI layer - Groq models are used for text parsing, voice transcription, and vision-based receipt interpretation, with deterministic/local fallbacks where implemented.
- Observability layer - PRISM instrumentation records AI/pipeline calls for traceability and latency visibility.

Technology stack

Module / Feature	What the user can do	Implementation detail
Stack Frontend	Streamlit	Rapid Python UI, forms, expanders, dataframes, session state and custom CSS.
Stack Backend / application	Python	Repositories, use-case services, utility functions and navigation state.
Stack Database	PostgreSQL	Durable user-scoped expense, item, budget, friend, timeline, reminder and insight persistence.

Module / Feature	What the user can do	Implementation detail
Stack AI	Groq APIs	Text model, Whisper audio transcription, vision models for receipt extraction.
Stack OCR fallback	Tesseract + OpenCV	Local text extraction and image preprocessing when vision path is unavailable.
Stack Payments UX	UPI URI + QR	Sample deep links and QR codes with prefilled payee/amount; no payment processing.
Stack Observability	PRISM by Block Convey	AI and pipeline tracing from the application.
Stack Deployment	Render	Public live deployment URL for the Streamlit application.

5. AI & Intelligent Automation

HYBRID AI DESIGN

The application keeps financial facts deterministic and uses the language model primarily to interpret or explain those facts. This reduces the chance that a generative response becomes the source of truth for transaction totals.

5.1 Voice transcription

The Add Expense screen exposes an audio-input flow. The implementation sends audio to Groq Whisper when configured and falls back to SpeechRecognition when the primary service cannot be used. The resulting transcript is passed into the expense parser.

5.2 Natural-language expense parsing

The parser converts a sentence into structured fields including title, category, total amount, optional friend, and item lines. A lightweight heuristic fallback handles common student categories and amount extraction when an LLM response is unavailable.

5.3 Receipt / screenshot intelligence

Receipt scanning uses a three-tier strategy: vision-capable Groq models first; local Tesseract/OpenCV text extraction when necessary; and an LLM parser for OCR text. The structured result includes vendor/title, category, total amount, purchased items, and a confidence value.

5.4 Financial insights

The dashboard requests the latest persisted insight. The service can generate a hybrid insight from deterministic spending aggregates and then use the language model to turn those facts into student-friendly guidance. The UI includes a Refresh AI Insight control.

5.5 Reminders

Reminders are evaluated from current budget information, persisted, shown on the dashboard, and dismissible by the user. This allows budget warnings to remain visible without turning the app into a generic notification system.

AI safety / trust boundary

- The final expense record is created by the same deterministic database transaction path regardless of whether it originated from manual entry, voice, or OCR.
- AI output should always be treated as a suggested prefill until the student confirms the form.
- UPI features are sample/demo payment shortcuts only; they do not perform or confirm real financial transactions.
- API secrets belong in environment variables and should not be committed to source control.

6. Data Model & Persistence

The current implementation is organized around durable PostgreSQL persistence. The main application calls repository methods for expenses, budgets, friends, timeline, reminders, insights and users, with atomic expense saving for the complete record.

Entity	Purpose
Users / session context	Identifies the active student and provides the user scope applied to queries and mutations.
Friends	Stores named friend records, sample UPI IDs, and optional relationship/context notes.
Expenses	Stores the high-level transaction: date, title/merchant, category, amount, optional friend, note, and evidence metadata.
Expense items	Stores normalized purchase lines such as item name, quantity, unit price and line total.
Budgets	Stores daily and monthly budget limits and supports utilization calculations.
Timeline events	Append-only activity records with event type, timestamp and structured JSON-like payload.
Reminders	Persisted actionable warnings or nudges, including dismissal state.
Insights	Persisted latest AI/deterministic financial guidance.
Attachments	Stores evidence metadata associated with an expense; the UI treats receipt and screenshot as one evidence concept.

Atomic expense persistence

When the student saves an expense, the application passes the complete payload - header fields, item lines, optional friend, note, and attachment metadata - through a single save operation. The user-visible success state is shown only after the transaction succeeds; errors are surfaced as a rollback message.

Historical item visibility

The central persistence improvement is normalization of purchased items. History records are expandable, and the dashboard category sections can also drill down to the item rows that contribute to category spend. This turns a past expense from a single number into a useful mini-ledger.

7. User Interface & Interaction Design

- Wide Streamlit layout with persistent sidebar navigation.
- Custom card-based styling with navy, marigold, green, brick, paper and neutral tokens.
- Display typography separated from body text, with a monospaced style for money figures.
- Circular daily/monthly budget meters replace a plain numeric-only budget presentation.
- Expandable category and history sections avoid overwhelming the main dashboard while keeping item details one click away.
- A single Back action is centralized so every routed screen follows the same navigation contract.
- Contextual feedback is shown for OCR confidence, transaction commit success, and empty states.

Navigation model

The sidebar exposes six primary routes: Dashboard, Add Expense, History, Timeline, Budgets, and Friends. Navigation pushes the previous route onto a session-state stack. The Back button pops that stack; if there is nothing to pop, the application returns to the dashboard. This explicitly addresses the prototype back-button issue.

8. Feature-Level Acceptance Criteria

AC-01 Expense capture: A user can save a manual expense with title, amount, category, date and item lines; the record appears in History after refresh.

AC-02 AI capture: A voice or text description can prefill the standard expense form; the user can review before saving.

AC-03 OCR capture: A clear receipt image can be scanned and used to prefill vendor, amount, category and item lines, with a confidence value when available.

AC-04 Friend context: A user can add a named friend with a sample UPI ID and later retrieve that friend from the dashboard/friends screen.

AC-05 Budget tracking: Daily and monthly budgets can be set independently and reflected in the corresponding circular meters.

AC-06 Item history: Expanding a historical expense reveals item name, quantity, unit price and line total.

AC-07 Category drilldown: Expanding a dashboard category reveals the item rows contributing to that category.

AC-08 Timeline: Saving expenses and updating budgets/friends creates timeline events that persist beyond the current session.

AC-09 Reminders & insights: The dashboard displays persisted insight content and active reminders, with a refresh/dismiss interaction where supported.

AC-10 UPI demo: A friend can generate a sample QR and app-specific UPI deep links with a prefilled amount; the UI explicitly states that no real transaction is processed.

AC-11 Back navigation: From any non-dashboard route, Back returns to the prior route or dashboard deterministically.

AC-12 No roommate concept: Current user-facing relation management uses Friends rather than a roommate ledger model.

9. Deployment & Run Information

The public project is deployed as a Streamlit application at the live URL below. The repository README documents the Streamlit launch command and environment variables used by the AI, telemetry and database layers.

LIVE APPLICATION

hostel-pocket-breakingbytes.onrender.com

[Open the live application](https://hostel-pocket-breakingbytes.onrender.com)

SOURCE CODE

github.com/JavinKapur/Hostel_Pocket_BreakingBytes

[Open the GitHub repository](https://github.com/JavinKapur/Hostel_Pocket_BreakingBytes)

Environment configuration categories

- DATABASE_URL - PostgreSQL connection information.
- GROQ_API_KEY - AI text, voice and vision access when enabled.
- PRISMTRACE_API_KEY / PRISMTRACE_PROJECT_ID / PRISMTRACE_HOST - observability configuration.

Secrets should remain in deployment environment settings or a local .env file and must not be committed to Git.

10. Implementation Verification Notes

This document is a final workshop feature/technical summary based on the current public GitHub repository and its linked live deployment. The repository currently presents the HostelPocket implementation as a Streamlit + PostgreSQL application with PRISM observability and sample UPI deep links.

Area	Current status	Evidence / notes
Dashboard	Implemented	Daily/monthly meters, quick actions, category item drilldown, friend shortcuts, AI insight and reminders are present in the main app.

MAINTENANCE NOTE

The public repository still contains older/legacy helper code in addition to the newer repository/service-based main app. The user-facing feature catalogue above describes the current main Streamlit implementation, not those legacy paths.

11. Future Enhancement Scope

- Introduce a full login/sign-up experience with password hashing, session expiry, and explicit account switching.
- Move evidence files to managed object storage instead of treating attachment metadata as the primary persistence boundary.
- Add edit/delete expense workflows with corresponding timeline audit events.
- Add richer analytics such as weekly trend charts, category budget caps, and forecasted month-end spend.
- Allow students to create custom categories and recurring expenses.
- Add export to CSV/PDF and a one-click semester/month financial summary.
- Add automated scheduled reminder delivery beyond in-app reminders.
- Add automated tests for repository transactions, route stack behavior, budget calculations, parser validation and QR URI construction.
- Add stricter data validation and confidence-aware review states for AI/OCR extracted totals.

12. Conclusion

HostelPocket brings everyday hostel spending into one student-centered workflow: fast capture, durable history, item-level visibility, budget awareness, friend context, AI-assisted entry, and lightweight payment shortcuts. The current implementation also adds PostgreSQL persistence, deterministic navigation, reminders, timeline storage, and PRISM observability, giving the project a clear path from prototype to a more maintainable application.

PROJECT IDENTITY

Javin Kapur • 26BCE0024 • SCOPE (CSE Core) • HostelPocket

Project References

Live website: <https://hostel-pocket-breakingbytes.onrender.com>

GitHub repository: https://github.com/JavinKapur/Hostel_Pocket_BreakingBytes

Primary implementation files reviewed: hostelpocket_app.py, ai_services.py, upi_engine.py, README.md, plus the repository structure for database/repository/service/UI modules.